

Computer Simulation

Lesson 3: Flow Controls

Takumi Miyoshi, Taku Yamazaki

Special thanks to: Dr. Pham Thi Huong

Lesson 3

Flow Controls

- Selection statements
 - The **if**
 - The **switch**
- Loops
 - The **while** Loops
 - The **for** Loops
 - Nested Loops
- Break
- Practices

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

Selection statements

- Selection statements
 - How to make choices
 - Deciding whether statements are executed or not
 - How to choose between or among choices
- MATLAB has two basic selection statements:
 - The **if** statement
 - The **switch** statement

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

The **if** statement

General form	Example
if condition Commands end	num = input('Enter a random number: '); if num>0 disp('number is positive') end if num<0 num = abs(num) end

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

The if-else statement

General form	Example
if condition Commands else Commands End	num = input('Enter a random number: '); if num>0 disp('number is positive') else disp('number is negative') end

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

The if-elseif-else statement

General form	Example
if condition1 Commands1 elseif condition2 Commands2 ... elseif condition $n-1$ Commands $n-1$ else Commands n end	>>num = input('Enter a random number: '); if num>0 disp('number is positive') elseif num<0 disp('number is negative') else disp('number is zero') end

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

Nested if-else statement

General form	Example
if condition1 Commands1 else if condition2 Commands2 else Commands3 end end	>>num = input('Enter a random number: '); if num>0 disp('number is positive') else if num<0 disp('number is negative') else disp('number is zero') end end

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

Comparison

- Single “if-end” clause and “nested if-else” or “if-elseif” clauses → which is more efficient

```
num = input('Enter a random number: ');  
if num>0  
    disp('number is positive')  
end  
if num<0  
    disp('number is negative')  
end  
if num==0  
    disp('number is zero')  
end
```

Takumi Miyoshi / Taku Yamazaki

```
>>num = input('Enter a random number: ');  
if num>0  
    disp('number is positive')  
else  
    if num<0  
        disp('number is negative')  
    else  
        disp('number is zero')  
    end  
end
```

Lesson 3: Flow Controls

The **switch** statement

General form	Example
switch expression case case_exp1 Commands1 ... case case_exp <i>n</i> -1 Commands <i>n</i> -1 otherwise Commands <i>n</i> end	num = input('Enter a random number: '); switch sign(num) case 1 disp('number is positive') case -1 disp('number is negative') otherwise disp('number is zero') end

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

Relational expressions

Operator	Meaning	Result
>	Greater than	Logical true: 1 Logical false: 0
<	Less than	
>=	Greater than or equal to	Example: a=3>4 → a=? a=3==3 → ?
<=	Less than or equal to	
==	Equality	
~=	Inequality	
	Or for scalars	
&&	And for scalars	
~	not	

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

Loops

- Two basic types of loops
 - Counted loops: **for**
 - Conditional loops: **while**
- Counted loops
 - Repeat statements a specified number of times
 - When it is known how many times the statements are to be repeated
- Conditional loops
 - Repeat the statements until the condition is satisfied
 - When it is unknown how many times repeated

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

The **for** Loop

General form	Example
for variable=range Commands end	for i=1:5 disp(i) end

The easiest way to specify the range of values is to use the colon operator “:” x= i:j or x = colon(i,j)
 x =[i, i+1, ..., j];

Takumi Miyoshi / Taku Yamazaki

Lesson 3: Flow Controls

Application of the **for** Loop

- Summation

$$S = 1+2+3+4+5+\dots+n = \sum_{i=1}^n i$$

S=0; for i=1:n; S=S+i; end; S

- Product

$$P = 1*2*3*\dots*n = \prod_{i=1}^n i = n! \text{ (factorial of } n\text{)}$$

P=1; for i=1:n; P=P*i; end; P

Built-in function

- Calculate summation; **sum**

$$S = 1+2+3+4+5+\dots+n = \sum_{i=1}^n i$$

S = sum(1:n)

- Product: **prod, factorial**

$$P = 1*2*3*\dots*n = \prod_{i=1}^n i = n! \text{ (factorial of } n\text{)}$$

P = prod(1:n); P = factorial(n)

Sum and Prod with Vectors

- Calculate summation: **sum**

S = sum([2 4 6])

- Product: **prod**

P = prod([2 4 6])

- Example: $S=1^2 + 2^2 + 3^2 + \dots + n^2$

S=0; for i=1:n; S=S+i.^2; end; S

i=1:n; S=sum(i.^2)

The **while** Loop

General form	Example
while condition Commands end	i=0; S=0; while S<1989 S=S+i; i=i+1; end S,i

Nested Loops (1)

- Nested **for**
- Nested **while**

General form

```
for var1=range1
  for var2=range2
    commands
  end
end
```

General form

```
while condition1
  commands1
  while condition2
    commands2
  end
end
```

Nested Loops (2)

- Example: Calculate Sum of the matrix A
- A=[1,3,5;7,9,2;4,6,8]

```
S=0;
for i=1:3
  for j=1:3
    S=S+A(i,j);
  end
end
S
```

```
S=0;i=0;
while i<3
  i=i+1; j=0;
  while j<3
    j=j+1;
    S=S+A(i,j);
  end
end
S
```

The **break** statement

- **break** can terminate the execution of **while** or **for** loop.
 - In nested loops, **break** exits from the innermost loop only.
 - **break** cannot be defined outside of **for** or **while** loop. Use **return** in this context instead.
- Very useful statement in the loop statement

Example 1

- Generate random numbers and terminate the generation when you got the integer 1989, count the number of loops
- Hint
 - Random number: **rand**
 - Integer random number: **randi** or **fix**

Answer 1

```
count = 0
num = randi(2000); %generate integer from 1 to 2000
while 1 %always loop
    num = randi(2000);
    count = count + 1;
    if num == 1989
        num, count, break
    end
end
```

Answer 2

```
count = 0
while 1 %
    num = fix(rand.*10000); %
    count = count + 1;
    if num == 1989
        num, count, break
    end
end
```

Application

- Searching: looking for a value in a list or in a vector
- Sorting: process of putting a list in order
 - Highest to lowest: descending
 - Lowest to highest: ascending

Practice 1: Sorting

- Distribute a random integer vector with 10 elements, values is from 2 to 28
- Sort the vector in ascending order
- Random number: **rand**, **randi**

Practice 2

Given the vector $x = [1 \ 2 \ 3 \ 4 \ 5 \ 6]$, create a short set of commands that will

1. Add up the values of the elements (Check with **sum**.)
2. computes the sine of the given x-values (should be a vector)

Practice 3

Create a 3 x 6 array of random numbers (use **rand**). Move through the array, element by element, and set any value that is less than 0.5 to 0, and any value that is greater than (or equal to) 0.5 to 1.

Practice 4

Write a script that asks for a temperature (in degrees Fahrenheit) and computes the equivalent temperature in degrees Celsius. The script should keep running until no number is provided to convert. [Check with the function **input()**, **isempty()**, and **num2str()**]

Thank you very much for your attendance!